

AI Property Development Platform Plan

Internal strategy, build phases, technology and delivery plan

Prepared as an internal working plan for scoping, closing and delivering Monali Kamffer's AI-assisted property development platform.

Version: 1.0 | Date: May 27, 2026

Working position

Build the platform in controlled phases. Start with a practical MVP that proves the highest-value workflow: upload property documents, extract structured intelligence, capture a house brief, generate site-aware concept options, estimate rough cost, and export a professional concept pack. Do not promise final architectural, engineering, municipal or QS-grade outputs in version 1.

Internal note	How to use this document
Purpose	This is for your own planning before sending client-facing scope or pricing.
Client-facing version	Share only a simpler MVP summary, flowchart and milestone/hour estimate after we clean it up.
Main risk	The vision is very large. Close with a paid discovery milestone first, then propose the MVP build.

Table of Contents

1. Executive direction
2. Client problem and product promise
3. Proposed platform workflow
4. MVP scope
5. Full phased roadmap
6. Technical architecture
7. AI and layout-generation strategy
8. Data model
9. Technology stack
10. Delivery workflow
11. Risks and controls
12. Client-closing strategy
13. Appendix: Questions and checklists

1. Executive direction

The project should be framed as an AI-assisted decision and concept platform for property developers, not as an AI replacement for architects, engineers, town planners or quantity surveyors. The first commercially useful product is a tool that turns documents and developer inputs into early-stage feasibility intelligence, concept layouts, design direction, rough cost ranges and exportable reports.

- Start with one real pilot use case, ideally a single stand or two spec houses from Greens Boutique Estate.
- Build around the current AI Studio prototype only as visual/product inspiration, not as the final technical base unless she can export or provide usable source code.
- Keep the first paid commitment small: discovery, current-app review, MVP scope and roadmap.
- Then move into a funded MVP build with clear deliverables and human review checkpoints.

Strategic principle

If we try to build the full vision immediately, the scope will become impossible to estimate. If we build only a simple dashboard, the client may feel we missed the real vision. The right middle path is a working Single Stand AI Concept Generator supported by document intelligence, brief builder, cost range and PDF export.

Recommended positioning

Avoid saying	Say this instead	Reason
AI will create final architecture drawings.	AI will generate early-stage concept layouts and decision reports.	Safer, more realistic, and legally cleaner.
We can automate all approvals.	We can extract rules and flag compliance risks for review.	Municipal and legal interpretation needs human validation.
We can build everything in one MVP.	We will prove the core workflow first, then expand into estate-wide intelligence.	Controls scope and builds trust.
The layout engine will be purely LLM-based.	The layout engine will combine structured rules, geometry, scoring and AI explanation.	Pure LLM is weak for geometry; hybrid approach is stronger.

2. Client problem and product promise

The client's document describes a South Africa-specific platform for land, layout, design, estate, climate, costing and build intelligence. The strongest core promise is to help developers answer: what should we build, where should it sit on the stand, and how can the concept be built more efficiently and commercially.

Problem statement

Developers currently hold project information across PDFs, zoning documents, survey drawings, site development plans, estate guidelines, emails, reports and spreadsheet-like decisions. Turning that information into a practical house concept, cost view and development decision pack is slow, fragmented and dependent on multiple experts at an early stage.

Solution statement

Build a web platform where a developer creates a project, uploads documents, verifies extracted constraints, enters a structured house/development brief, and receives 2-3 site-aware concept options with layout logic, visual direction, cost range, buildability notes and exportable reports.

Primary users

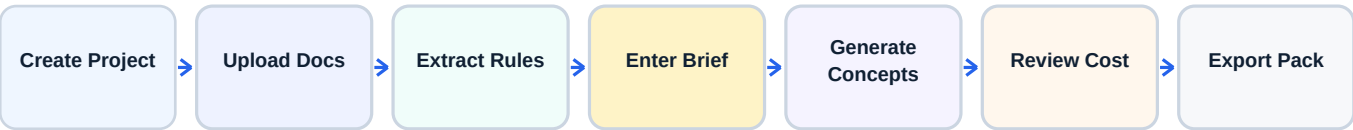
User	Main need	How the platform helps
Developer / landowner	Feasibility, cost and design direction	Quickly compare what can be built on a stand and export concept packs.

User	Main need	How the platform helps
Estate planner	Stand-by-stand strategy	Classify stands, select spec house candidates and manage estate rules.
Architect / concept designer	Faster initial concept work	Receive structured site constraints and AI-generated starting layouts.
Builder / QS	Early buildability and cost picture	Review rough quantities, complexity notes and cost sensitivity before detailed pricing.

3. Proposed platform workflow

The platform should guide a non-technical user through a simple wizard while running complex intelligence in the background. The first version should make the workflow feel easy even if the underlying inputs are messy.

High-level product flow



Core user journey

Step	What user does	What system does	Output
1. Create project	Choose Single Stand, Estate, Revamp or Portfolio project.	Creates project shell, folders, default checklist and data model.	Project workspace.
2. Upload documents	Add zoning, title deed, estate rules, survey, contours, SDP, reports, images.	OCR/parses files, chunks content, stores originals and extracted fields.	Structured data room.
3. Validate constraints	Reviews AI-extracted rules and confirms or edits them.	Turns rules into design constraints and warnings.	Validated project constraints.
4. Add stand/site details	Enter erf, coordinates, boundaries, road/access side, contour notes.	Creates stand geometry, buildable envelope and site assumptions.	Site intelligence snapshot.
5. Fill brief	Define rooms, levels, lifestyle, style, budget and priorities.	Converts brief to room graph and layout requirements.	Validated design brief.
6. Generate concepts	Clicks generate and compares options.	Produces 2-3 schematic plans, site placement, scoring and rationale.	Layout concept options.
7. Add visuals	Chooses visual direction or material style.	Creates exterior concept prompts/images and style cards.	Concept renders or image directions.
8. Estimate cost	Selects cost quality level and assumptions.	Calculates area, multipliers, external works and cost range.	Rough cost + buildability view.
9. Export	Selects what to include.	Generates PDF concept/feasibility pack.	Shareable report.

4. MVP scope

The MVP should prove the core value with one practical scenario: a developer uploads a stand/project package, completes a guided brief, and receives useful early-stage layout concepts, rough costs and a report. The MVP should be strong enough to feel real, but narrow enough to deliver safely.

Recommended MVP name

Single Stand AI Concept Generator

Module	MVP functionality	Implementation approach	Later expansion
Auth + workspace	Login, project list, project details, basic roles.	Next.js app + Clerk/Supabase Auth.	Teams, permissions, audit logs.
Data room	Upload PDFs/images/KML/DXF/DWG where possible; categorize docs.	Object storage + metadata + OCR queue.	Versioning, shared comments, external consultant portal.
Document intelligence	Summaries, extracted rules, risks, key decisions, searchable Q&A.;	OCR + LLM JSON extraction + vector search + validation UI.	Document comparison, clause conflict detection, stronger legal workflows.
Site/stand input	Manual stand details, boundaries, north, road side, slope notes, upload contour files.	Map UI + simple geometry fields + optional KML parsing.	Automated contour/terrain engine, PostGIS overlays, LiDAR/GIS sources.
Buildable envelope	Show conceptual setbacks, no-build zones and placement zones.	Rule engine based on validated inputs and manual overrides.	More automated zoning and estate guideline enforcement.
Brief builder	Rooms, levels, style, budget, priorities and relationship rules.	Guided form + JSON schema.	Reusable templates per estate house type.
Layout concepts	2-3 site-aware schematic options with labels, room blocks, site placement and rationale.	Hybrid LLM + constraint/rule engine + SVG/canvas rendering.	CAD/BIM export, terrain-aware massing, multi-unit optimization.
Visual concepts	Exterior concept prompts/images and material style cards.	Image generation API or prompt packs; mark as concept only.	3D rendering pipeline, facade variations, streetscape simulation.
Cost estimate	Rough area, rate/m2, complexity multiplier, external works, contingency.	Formula-based calculator with configurable SA assumptions.	Detailed BOQ database, QS export, live cost library.
Export center	PDF concept pack: inputs, extracted rules, layouts, cost and recommendations.	Puppeteer/React PDF or server-side PDF generator.	Board-ready reports, schedules, Excel exports, client branding.

MVP boundaries

- MVP outputs are concept-level and decision-support level, not construction drawings.
- Every AI-extracted legal, zoning or estate rule must be user-verifiable before being used as a hard constraint.
- The layout engine should generate schematic floor/site plans first. Final architectural drawings remain outside MVP.
- Cost outputs should be range-based and assumption-driven, not QS-certified BOQs.
- Use one pilot stand/project first. Do not try to automate every entity in the client’s portfolio from day one.

MVP deliverables

Deliverable	Definition of done
Working web app	Deployed app with login, project dashboard, data room and project workspace.
Document intelligence	Upload docs, run extraction, view structured fields, ask project-specific questions.
Brief builder	Structured form that stores room counts, priorities, style and constraints.
Concept generator	Generates 2-3 schematic concept options with site logic and rationale.
Cost + buildability	Displays cost range, complexity score, earthworks/retaining notes and assumptions.
Export pack	Downloads a clean PDF report for review/share.
Admin/config	Manage rate/m2 assumptions, output templates and project categories.

5. Full phased roadmap

The whole platform should be delivered as a sequence of value-based milestones. Each phase should leave the client with something usable, not just backend work.

Phase	Goal	Key work	Estimated time
0. Discovery + scope	Turn the broad idea into a buildable MVP.	Review AI Studio app, documents, target first use case, acceptance criteria, risks, pricing basis.	2-5 days
1. Foundation app	Create the base product shell.	Auth, dashboard, project CRUD, roles, storage, base database, deployment pipeline.	1-2 weeks
2. Data room + doc AI	Make uploads useful.	OCR/parsing, document categories, summaries, structured extraction, Q&A;, validation UI.	2-3 weeks
3. Site + rules intelligence	Convert project inputs into constraints.	Stand input, map view, buildable envelope, setbacks, access side, slope assumptions, constraint registry.	2-3 weeks
4. Brief builder	Capture design intent in structured form.	Room counts, levels, lifestyle, priorities, exact dimensions/free text, brief validation.	1-2 weeks
5. Layout engine v1	Prove site-aware concept generation.	Room graph, constraint prompts, layout candidates, scoring, SVG/canvas plan rendering, design rationale.	3-5 weeks
6. Cost + export	Turn concepts into decision packs.	Cost formula, complexity multipliers, preliminary BOQ categories, PDF export templates.	1-2 weeks
7. Visual concept layer	Add visual selling power.	Exterior prompts, image generation, style cards, material palette, render comparison.	1-2 weeks
8. Pilot QA + handover	Stabilize with a real project.	Test on one real stand, fix extraction/layout issues, docs, training, launch.	1 week
Phase 2	Expand from one stand to estate-level intelligence.	Parent-child projects, stand classification, house-type templates, estate dashboard, stronger climate and security layers.	6-10 weeks
Phase 3	Advanced automation and professional exports.	CAD/BIM export, deeper BOQ, municipal readiness, multi-unit optimization, services integrations.	Ongoing

Suggested first contract structure

Milestone	Client-friendly name	Output	Why this helps close
M1	MVP Planning + Technical Roadmap	Reviewed current app, locked MVP scope, feature matrix, architecture, estimate and next milestones.	Small commitment, low risk, shows expertise before big build.
M2	Foundation + Data Room	Working project dashboard, uploads, document storage and initial AI summaries.	Creates tangible progress quickly.
M3	Brief + Site Intelligence	Brief builder, stand input and buildable-envelope/rules flow.	Moves toward core product instead of generic dashboard.
M4	Concept Generator v1	2-3 schematic layout options with rationale and scoring.	Main wow feature.
M5	Cost + Export + Pilot	Cost range, PDF pack, pilot fixes and deployment handover.	Completes MVP value loop.

6. Technical architecture

Use a modular architecture. Keep the web app fast and polished, but move heavy AI, OCR, geospatial and layout-generation work into background jobs and Python services.

Recommended system architecture



Architecture layers

Layer	Responsibility	Recommended tech
Frontend	Dashboard, data room, forms, maps, plan viewer, reports.	Next.js, TypeScript, Tailwind CSS, shadcn/ui, React Query/Zustand.
Application API	Project CRUD, permissions, file metadata, AI job orchestration.	Next.js API routes/server actions for lightweight logic; FastAPI for AI/geospatial services.
Database	Projects, documents, extracted fields, constraints, briefs, concepts, costs.	PostgreSQL with PostGIS; Prisma or Drizzle; pgvector if keeping vector search in Postgres.
File storage	Original files, extracted pages, generated images, reports.	Supabase Storage, AWS S3, Cloudflare R2 or GCP Cloud Storage.
Queue/worker	OCR, embeddings, document extraction, layout generation, PDF rendering.	Celery/RQ + Redis, BullMQ/Upstash, Cloud Tasks or serverless queues.
Document AI	OCR, summary, structured rule extraction, RAG Q&A.;	OpenAI/Gemini/Anthropic + OCR service + embeddings + JSON schemas.
Geospatial	Boundaries, envelopes, contours, slopes, overlays.	PostGIS, Shapely, GeoPandas, pyproj, rasterio/GDAL, Mapbox/MapLibre.
Layout engine	Generate and score concept layouts.	Python service using geometry rules + LLM + SVG/canvas output.
Reporting	Generate concept packs and feasibility summaries.	Puppeteer/Playwright HTML-to-PDF, React-PDF, or ReportLab.
Deployment	Reliable app hosting and secure background jobs.	Vercel for frontend; Render/Fly/AWS/GCP for Python workers; Sentry + PostHog.

Why not build everything inside one Next.js app?

- Layout generation, OCR, contour processing and document parsing are long-running tasks. They need queues/workers.
- Python has stronger geospatial and geometry libraries than Node for this use case.
- Keeping AI services modular makes it easier to swap models, add CAD parsing or move to cloud services later.

7. AI and layout-generation strategy

The layout engine is the core product, but it must be built carefully. The safest strategy is a hybrid engine: deterministic geometry and scoring for spatial correctness, LLMs for interpretation, option naming, rationale and user-friendly explanations.

Document intelligence pipeline

Document AI pipeline



Stage	How it works	Important control
OCR/text extraction	Extract text from PDFs and scans; keep page references and confidence.	Flag low-confidence pages for manual review.
Classification	Identify zoning, title deed, estate rules, survey, SDP, cost report, investor report etc.	User can correct document type.
Structured extraction	Use JSON schemas to extract setbacks, height limits, coverage, risks, dates and decisions.	Never silently treat AI extraction as final truth.
Vector search/RAG	Allow project Q&A; grounded in uploaded documents.	Answer with document citations/page references in the app.

Stage	How it works	Important control
Constraint registry	Converted extracted rules into usable constraints for layout and warnings.	Validated/overridden by user.

Layout engine v1 pipeline

Layout generation pipeline



Component	MVP approach	Quality requirement
Site model	Boundary polygon, north, road/access side, slope notes, buildable envelope.	Must know where the house can and cannot sit.
Rules	Validated zoning/estate rules plus manual assumptions.	Every hard constraint traceable to source/user entry.
Brief	Structured rooms, relationships, levels, lifestyle and priorities.	Needs dropdowns and free text; avoid only open prompt.
Room graph	Represent rooms as nodes with desired adjacency and separation.	Kitchen to scullery/garage/patio, living to outdoor, private bedroom zones etc.
Generation	Create several layout strategies using constraints + templates + LLM suggestions.	Multiple options, not one answer.
Scoring	Score sunlight, access, privacy, circulation efficiency, slope/earthworks, build complexity.	Expose why each option scored well/poorly.
Rendering	Draw schematic floor/site plans as SVG/canvas with labels and dimensions when possible.	Readable and useful; not final CAD.

Key engineering rule

Do not ask an LLM to directly invent a complete floor plan as plain text and trust it. Instead, make the LLM propose strategies, then convert/validate those strategies through structured geometry, room graphs, rules and scoring. The final output should be machine-checkable.

8. Data model

The database should support both single-stand projects and future estate-level parent/child projects. Keep the schema flexible but structured enough that AI outputs can be versioned and compared.

Entity	Stores	Notes
users	User profile, auth reference, role.	Use auth provider for credentials.
organizations	Client company/workspace.	Useful if Monali later invites team/consultants.
projects	Project name, type, location, status, parent project.	Supports Single Stand, Estate, Revamp, Portfolio.
stands	Erf/stand number, boundary, size, road side, slope, parent estate.	PostGIS geometry when ready.
documents	File metadata, type, status, storage URL.	Original files never overwritten.
document_chunks	Extracted text, page refs, embeddings.	Supports project Q&A.;
extracted_fields	Zoning, title, estate rules, decisions, constraints.	Include confidence + verified status.
constraints	Setbacks, height, coverage, privacy, access, style rules.	Used by layout engine.

Entity	Stores	Notes
briefs	Room counts, levels, style, priorities, budget, relationship rules.	Versioned; concepts link to a brief version.
concepts	Generated layout option, scores, rationale, SVG/image URLs.	Versioned; user can compare options.
cost_estimates	Area, rates, multipliers, allowances, totals.	Assumptions should be editable.
reports	PDF export settings and generated files.	Keep history of exported packs.
tasks_decisions	Client decisions, follow-ups, board actions.	Useful for portfolio/project management.

9. Technology stack

Recommended stack should balance speed, AI/geospatial capability, maintainability and Upwork-friendly delivery. We should use technologies we can confidently deliver with, while leaving room for enterprise-grade integrations later.

Area	Recommended choice	Reason
Frontend	Next.js + TypeScript + Tailwind + shadcn/ui	Fast to build polished SaaS UI; good for dashboards/forms/report previews.
Backend app	Next.js API + Python FastAPI service	Next.js for app operations; Python for AI/geospatial/layout engine.
Database	PostgreSQL + PostGIS	Handles relational app data plus future spatial queries.
ORM	Prisma or Drizzle	Type-safe DB access for app layer.
Vector search	pgvector first; Pinecone/Weaviate later if needed	Simple MVP deployment, can scale later.
Storage	Supabase Storage / S3 / Cloudflare R2	Secure storage for documents, images, exports.
Auth	Clerk, Supabase Auth or Auth0	Faster secure login than custom auth.
AI models	OpenAI / Gemini / Anthropic via adapter	Use model abstraction so we can switch based on cost/quality.
OCR	MVP: PDF text extraction + OCR fallback; stronger: Google Document AI/AWS Textract	Good enough first, more accurate later for scanned legal docs.
Maps	Mapbox/MapLibre or Google Maps	Map-based stand location and overlays.
Geospatial	PostGIS, Shapely, GeoPandas, pyproj, GDAL/rasterio	Industry-standard Python tools for geometry and terrain.
CAD parsing	ezdxf + OGR/GDAL where feasible	DXF support in MVP can be partial; DWG may need conversion layer.
Background jobs	Redis + Celery/RQ or BullMQ/Upstash	AI and document tasks should not block UI.
PDF export	Playwright/Puppeteer HTML-to-PDF or React-PDF	Easier to match client-facing report design.
Monitoring	Sentry + basic usage analytics	Catch failures in AI jobs and production app.

Model/cost strategy

- Use cheap/fast models for classification and summaries; use stronger models only for layout reasoning and complex extraction.
- Cache extraction results. Do not re-run costly OCR/LLM jobs unnecessarily.
- Add usage tracking per project/document so the client understands AI cost drivers.
- Use structured JSON outputs and validation to reduce hallucinations and rework.

10. Delivery workflow

A clear delivery process will help close the client and avoid scope chaos. Every major phase should produce a demo and an acceptance checkpoint.

Stage	Process	Output
Discovery	Meet client, confirm pilot project, collect files, review AI Studio prototype, identify true MVP success criteria.	Scope doc, feature matrix, architecture, milestone plan.
Design	Wireframe dashboard, data room, brief builder, concept result screens and export pack.	Clickable prototype or Figma screens.
Build	Develop features in milestone branches; weekly demos; keep backlog visible.	Working app increments.
AI tuning	Test on real docs, improve prompts/schemas, add validation and fallbacks.	Stable extraction and generation workflows.
Pilot	Run one real stand/project through the complete flow.	Pilot report and bug/quality list.
Launch	Deploy, setup backups, docs and handover training.	Production-ready MVP.
Expand	Use feedback to plan estate dashboard, better BOQ, CAD, climate and security modules.	Phase 2 roadmap.

Internal team responsibilities

Role	Responsibilities
Harshil	Lead architecture, AI workflow, backend/data model, client strategy, technical decisions, Upwork communication support.
Dhruvit	Frontend app, UI execution, forms, dashboards, reports, client demos, implementation support.
Optional expert later	Architecture/town-planning/QS advisor to validate outputs once MVP has traction.

Potential MVP hour range

Workstream	Conservative MVP hours	Notes
Discovery + scope	12-30	Depends on meeting depth and current app review.
UI/UX + screens	35-60	Dashboard, data room, brief builder, concept viewer, reports.
Foundation app	45-80	Auth, DB, storage, project CRUD, deployment.
Document intelligence	60-110	OCR, extraction, validation, Q&A, embeddings.
Site/rules intelligence	45-90	Manual + partial automated site constraints.
Layout generator v1	80-160	Largest uncertainty. Schematic first, not CAD.
Cost + export	35-70	Formula engine + PDF pack.
QA + pilot + handover	25-50	Testing on real project data.
Total realistic range	337-650	Use discovery to narrow before quoting fixed build.

11. Risks and controls

This project can become huge. The key is to control risk by narrowing the first use case, validating AI outputs, and making assumptions explicit.

Risk	Why it matters	Control
Scope explosion	Client vision includes documents, layouts, cost, climate, estate, BOQ, security and reports.	Start with one pilot workflow and separate Phase 1/2/3.
AI hallucination	Legal/zoning/cost outputs can be wrong if unverified.	Use structured extraction, confidence, page references, and user validation.
Layout quality	Poor concepts will make the platform feel fake.	Hybrid engine, scoring, templates, human review and clear concept-only label.
CAD/contour complexity	DWG/DXF/KML/PDF parsing varies widely.	MVP supports manual override and partial parsing; advanced CAD later.
Cost accuracy	Build costs vary by region, material and market.	Use configurable assumptions and range-based estimates.

Risk	Why it matters	Control
Legal/professional liability	Outputs might be mistaken for official approval-ready work.	Add disclaimers: advisory concept only, professional validation required.
New Upwork client	No hiring/payment history.	Use small funded milestones, no free deep implementation, keep all payments on Upwork.
Confidential docs	Client files may contain business/legal info.	Access controls, secure storage, no public sharing, optional data retention policy.

12. Client-closing strategy

The client seems warm and has a real business context, but she is likely non-technical and new to Upwork. The closing strategy should reduce uncertainty for her while protecting you from unpaid scope discovery.

Recommended next step with Monali

Propose a small paid discovery/planning milestone before any build commitment. The milestone should create: MVP scope, simple architecture, feature list, user flow, milestone estimate, and first pilot plan. After that, propose the build milestone.

What to send	Why
A simple MVP summary PDF	Non-technical client can understand the product direction quickly.
A one-page flowchart	Shows the workflow visually: Upload -> AI understands -> Brief -> Concepts -> Cost -> Report.
An Excel/Google Sheet feature estimate	Helps her see what is included, rough hours and phase split.
Not the full internal plan	Too technical and may overwhelm her or give away too much unpaid strategy.

Best client-facing artifact recommendation

- Use a short 3-5 page PDF for Monali: vision, MVP workflow, included modules, excluded later modules, first milestone.
- Attach a simple spreadsheet only after she agrees to discuss budget/timeline. The sheet should show features, hours, priority and milestone grouping.
- Do not send a 20-page technical plan to her now. She needs clarity, not engineering depth.

13. Appendix: questions and checklists

Questions to ask before final estimate

- Can she provide the source/export of the AI Studio app, or is it only a visual prototype/reference?
- What is the first real pilot project: Greens spec house, one existing stand, or another development?
- Which files can she provide for the pilot: zoning, title, estate guidelines, survey, contour, SDP, reports, images?
- What exact output does she expect from MVP: report only, schematic floor plan, exterior image, cost range, or all of these?
- Should users be internal only at first, or will external architects/builders/clients log in?
- What level of accuracy is acceptable for MVP cost estimates? Does she have preferred South African rate/m2 assumptions?
- Does she want the system to use her existing branding and report style from JJP Invest?
- What is her budget range and preferred timeline?
- Who will validate architectural/zoning/cost assumptions on her side?

MVP acceptance checklist

- A user can create a project and upload documents without developer help.
- The system extracts a useful project summary, constraints and risks from uploaded files.

- The user can verify or edit extracted constraints before generation.
- The brief builder captures room counts, levels, priorities, style and budget assumptions.
- The system generates at least 2-3 concept options with site placement and rationale.
- The concept output includes cost range and buildability notes.
- The user can export a clean concept PDF pack.
- All outputs are clearly marked as concept/advisory and requiring professional validation.

Internal final recommendation

Close with discovery first. Build the MVP around one real stand/spec-house workflow. Use a hybrid AI + rules + geometry engine, not pure prompt-based generation. Keep Phase 1 tight: document intelligence, brief builder, site-aware schematic concepts, rough cost and PDF export. Save estate masterplan automation, CAD/BIM, advanced BOQ and municipal readiness for later phases.

Source basis

This plan was prepared from the uploaded AI Development Platform blueprint, the Q1 JJP Invest report, and the Upwork chat context provided in screenshots. It is an internal planning document and should be simplified before sharing with the client.